

Relazione Tecnica - Progetto di Sistemi Distribuiti

“Prodotti in Vendita”

Corso di Laurea Magistrale in Informatica
Dipartimento di Matematica e Informatica

Giuliano Spata

Anno Accademico 2025/2026

Indice

1	Introduzione	3
2	Descrizione dei requisiti	3
2.1	Requisiti architetturali	3
2.2	Requisiti funzionali	3
2.3	Requisiti di sicurezza	4
3	Progettazione del sistema	4
3.1	Architettura a livelli	4
3.2	Remote Facade	4
3.3	Data Transfer Object (DTO)	5
3.4	Session State	6
3.5	Request Batch	7
3.6	JWT e controllo degli accessi	9
4	Dettagli di implementazione	11
4.1	Caricamento dei dati	11
4.2	Prodotti con rating migliore	12
4.3	Prodotto con prezzo migliore	12
4.4	Prodotti simili	12
4.5	Recensioni	13
4.6	Raggruppamento per fasce di prezzo	13
4.7	Ricerca e categorie	13
4.8	Modifica del prezzo	13
5	Elenco delle API	14
6	Esempio di utilizzo	14
6.1	Login	14
6.2	Richiesta protetta	14
6.3	Request Batch	15

7	Test e avvio	15
8	Limiti e possibili miglioramenti	15
9	Conclusioni	16

1 Introduzione

Il progetto realizza un servizio REST per la consultazione e la gestione di un catalogo di prodotti. L'applicazione è sviluppata in Java 17 con Spring Boot 3.2.0 e utilizza Spring Web per l'esposizione delle API, Spring Security per il controllo degli accessi, JWT per l'autenticazione stateless e Apache Commons CSV per il caricamento dei dati.

Il sistema lavora su quattro dataset CSV relativi alle categorie Skincare, Fashion e Laptops. I dati vengono caricati in memoria all'avvio e possono essere interrogati attraverso operazioni di ricerca, classificazione per rating, selezione del prezzo migliore, individuazione di prodotti simili, consultazione delle recensioni e raggruppamento per fasce di prezzo.

Non è presente un frontend dedicato. Le API possono essere utilizzate e testate tramite un client HTTP, per esempio Thunder Client.

2 Descrizione dei requisiti

2.1 Requisiti architetturali

Il sistema deve:

1. esporre API lato server attraverso una Remote Facade;
2. separare accesso ai dati, logica applicativa e gestione delle richieste HTTP;
3. utilizzare DTO per controllare i dati trasferiti al client;
4. aggregare più operazioni correlate in una singola Request Batch;
5. autenticare gli utenti tramite token JWT;
6. autorizzare le operazioni di scrittura in base al ruolo dell'utente.
7. mantenere una cronologia delle ricerche associata alla sessione utente;

2.2 Requisiti funzionali

Le principali funzioni offerte sono:

- elenco delle categorie disponibili;
- ricerca dei prodotti attraverso una parola chiave;
- restituzione dei cinque prodotti con rating migliore in una categoria;
- individuazione del prodotto con prezzo minore in una categoria;
- ricerca di prodotti simili della stessa categoria;
- recupero delle recensioni associate a un prodotto;
- raggruppamento dei prodotti in fasce di prezzo;
- aggiunta di una recensione da parte di un revisore;
- modifica del prezzo da parte di un amministratore;
- restituzione batch di prodotti migliori, prezzo migliore e prodotti simili;
- consultazione della cronologia delle ricerche della sessione.

2.3 Requisiti di sicurezza

Le operazioni GET sui prodotti sono pubbliche. Le operazioni di modifica richiedono invece autenticazione e ruoli specifici:

- **REVISORE**: può aggiungere recensioni;
- **AMMINISTRATORE**: può modificare i prezzi;
- **UTENTE**: rappresenta un utente autenticato senza permessi di scrittura speciali.

Nella configurazione attuale l'utente amministratore **prof** possiede direttamente anche il ruolo **REVISORE**, così può eseguire entrambe le operazioni.

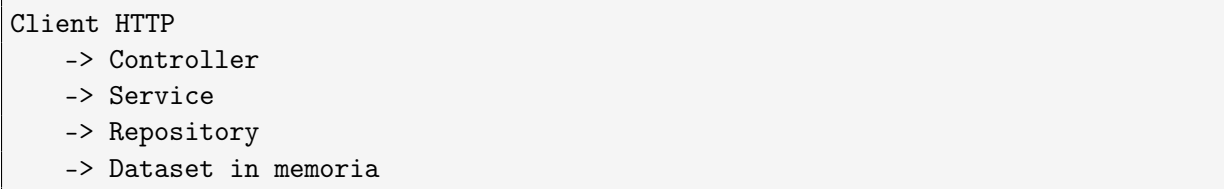
3 Progettazione del sistema

3.1 Architettura a livelli

Il progetto è diviso nei seguenti package:

- **controller**: espone le API REST e definisce le richieste HTTP;
- **service**: contiene la logica applicativa;
- **repository**: carica e rende disponibili i dati;
- **model**: contiene il modello interno e i DTO;
- **security**: gestisce autenticazione, token e autorizzazione.

Il flusso principale di una richiesta è:



La risposta segue il percorso inverso e, nella maggior parte degli endpoint, i modelli interni vengono convertiti in **ProductDTO** prima di essere serializzati in JSON.

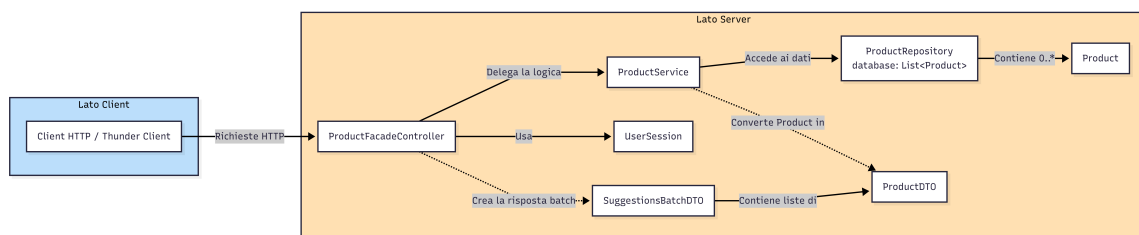


Figura 1: Diagramma UML dell'architettura generale

3.2 Remote Facade

La classe **ProductFacadeController** implementa il ruolo di Remote Facade. Espone un punto di accesso unico sotto il percorso:

```
/api/products
```

Il client non deve conoscere `ProductService`, `ProductRepository` o il formato dei file CSV. Comunica soltanto con la facade attraverso un insieme di endpoint REST. Il controller delega al service la logica applicativa e mantiene quindi separata la gestione HTTP dall'elaborazione dei dati.

Questa soluzione riduce l'accoppiamento tra client e struttura interna del server e permette di modificare service o repository senza cambiare il contratto HTTP, purché le API rimangano compatibili.

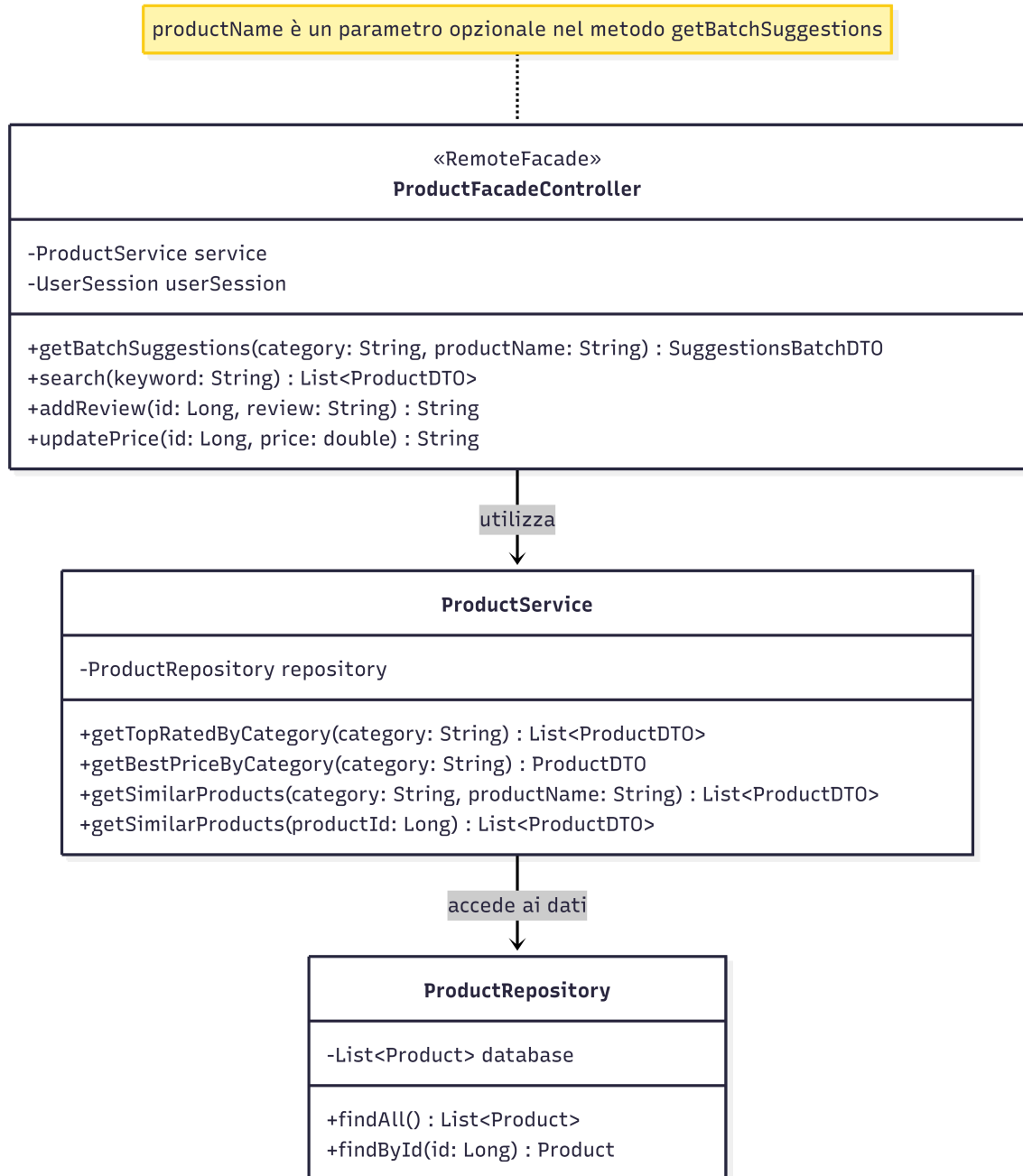


Figura 2: Diagramma UML del pattern Remote Facade

3.3 Data Transfer Object (DTO)

`Product` è il modello dati interno e contiene:

- identificativo;
- nome;
- categoria;
- prezzo;
- rating;
- recensioni.

`ProductDTO` è invece l'oggetto trasferito al client e contiene soltanto:

- identificativo;
- nome;
- categoria;
- prezzo;
- rating.

La conversione viene centralizzata nel metodo privato `toDTO` di `ProductService`. In questo modo il client non dipende direttamente dal modello interno e le recensioni possono essere recuperate attraverso un endpoint dedicato.

Un'eccezione controllata è l'endpoint `GET /api/products/{id}`, che restituisce direttamente `Product`. Questo metodo non è pensato come endpoint pubblico di consultazione, ma come endpoint ad hoc riservato agli amministratori per attività di debug e ispezione dello stato interno del prodotto, incluse le recensioni.

3.4 Session State

Il pattern Session State è implementato dalla classe `UserSession`, annotata con:

```
@Component
@SessionScope
```

Ogni sessione HTTP riceve quindi una propria istanza logica di `UserSession`. La classe conserva una lista di parole chiave cercate e impedisce l'inserimento di duplicati consecutivi.

`ProductFacadeController` utilizza questa classe in due punti:

- `GET /search` aggiunge la keyword alla cronologia;
- `GET /my-history` restituisce la cronologia corrente.

È importante distinguere questo stato applicativo dalla sicurezza JWT. `SessionCreationPolicy.ST` impedisce a Spring Security di salvare l'autenticazione nella sessione HTTP, mentre `UserSession` usa la sessione soltanto per la cronologia delle ricerche. Il token e la cronologia hanno quindi responsabilità differenti.

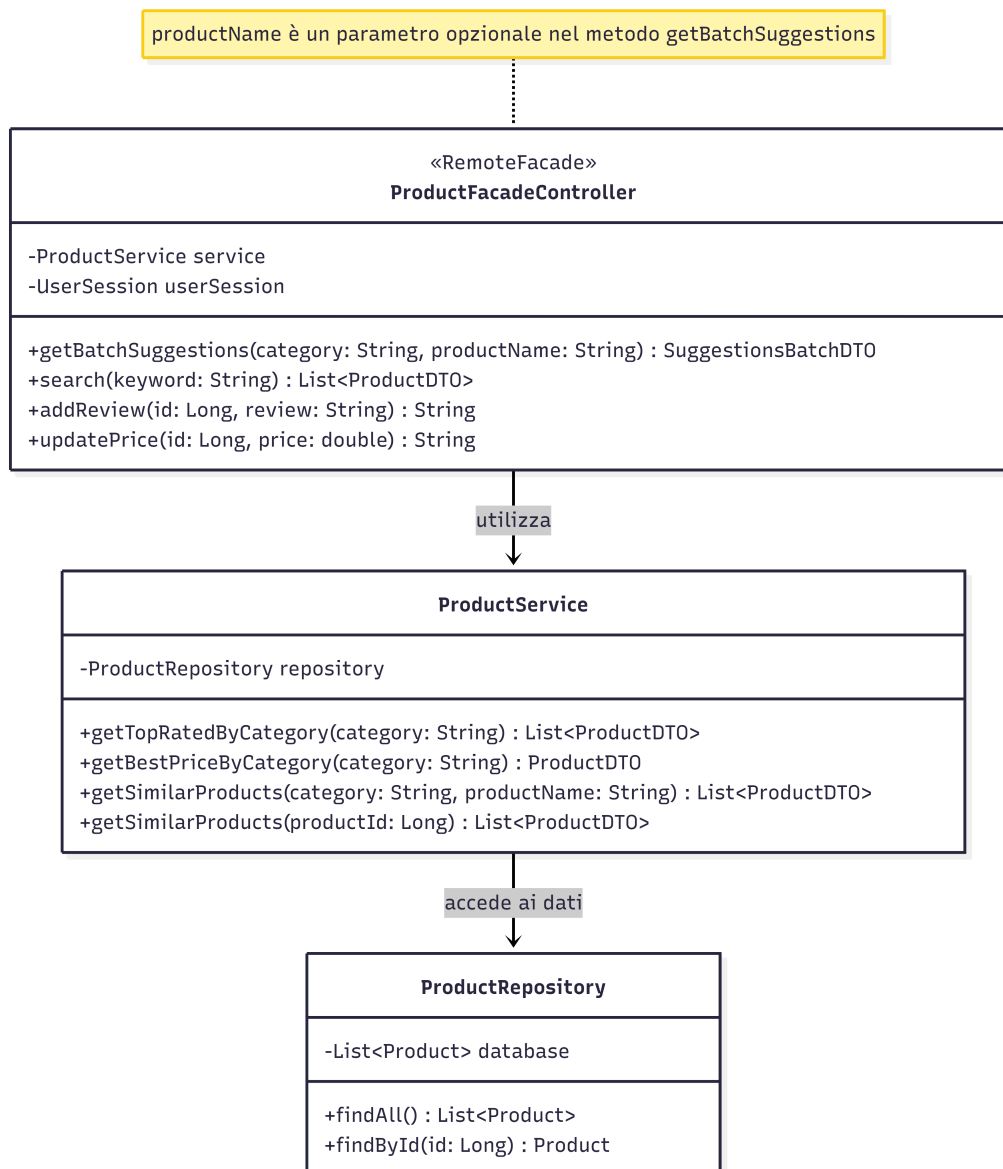


Figura 3: Diagramma UML del pattern Session State

3.5 Request Batch

Il pattern Request Batch è implementato con `SuggestionsBatchDTO` e con il metodo `getBatchSuggestions` di `ProductFacadeController`.

L'endpoint permette anche di effettuare una richiesta con due parametri:

- `category`, usato per limitare la ricerca a una categoria di prodotti;
- `productName`, usato per calcolare i prodotti simili rispetto a un nome di riferimento.

Un esempio di richiesta con entrambi i parametri è:

```
GET /api/products/suggestions?category=Laptops&productName=Apple%20MacBook%20Pro
```

La richiesta esegue tre elaborazioni:

1. seleziona i prodotti con rating migliore nella categoria;
2. individua il prodotto con prezzo minore nella categoria;

3. cerca prodotti della categoria simili al nome specificato.

I risultati vengono aggregati in un'unica risposta:

```
{
  "topRated": [],
  "bestPrices": [],
  "similar": []
}
```

Il parametro `category` viene utilizzato da tutte le elaborazioni, mentre `productName` influenza soltanto la lista `similar`. Senza Request Batch il client dovrebbe inviare tre richieste HTTP distinte. L'aggregazione riduce i round trip tra client e server e fornisce un risultato coarse-grained.

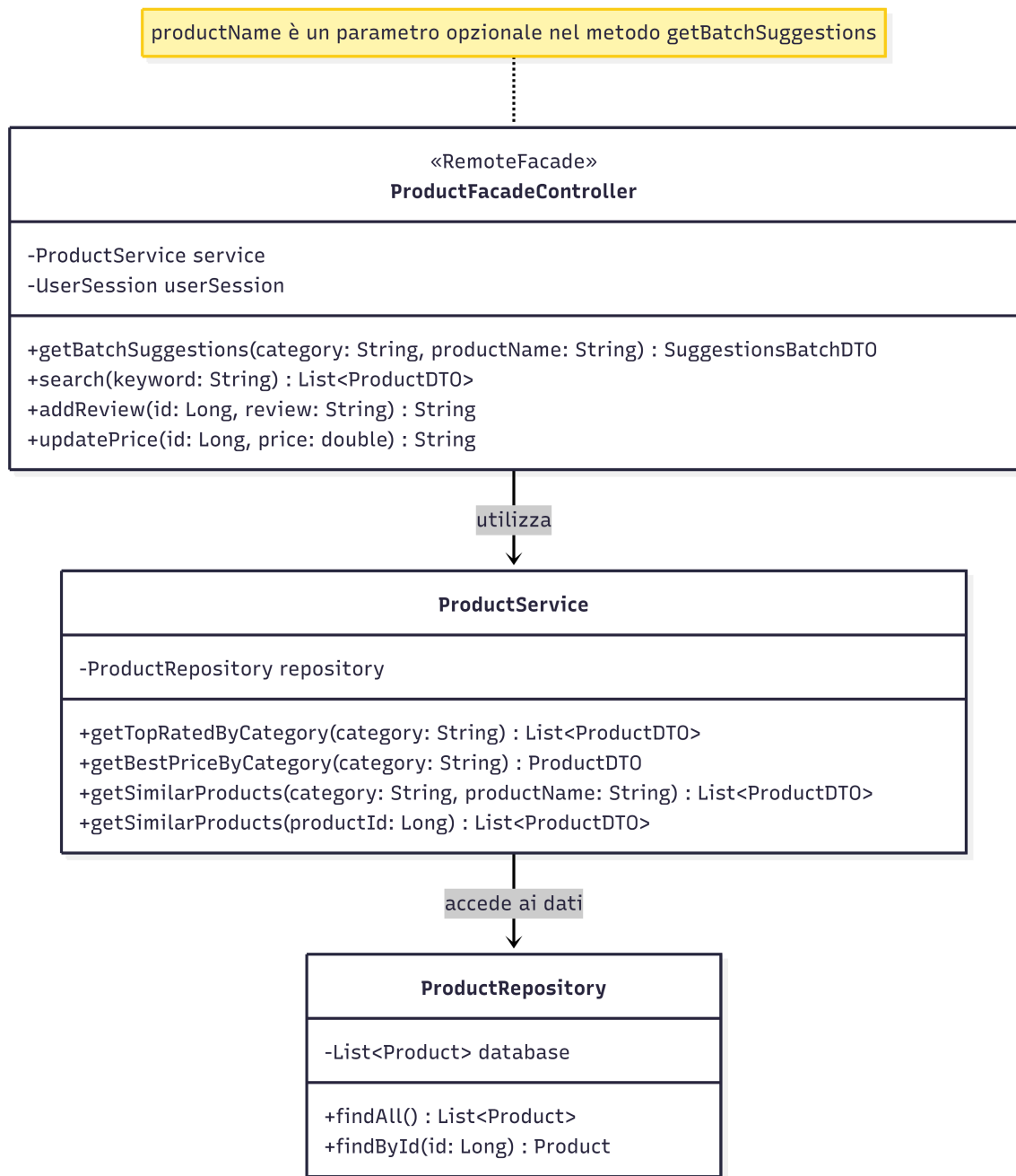


Figura 4: Diagramma UML di DTO e Request Batch

3.6 JWT e controllo degli accessi

Il login è esposto da:

POST /api/auth/login

Il client invia un oggetto `AuthRequest` con username e password. `AuthController` autentica le credenziali tramite `AuthenticationManager`, carica l'utente con `UserDetailsService` e richiede a `JwtUtil` la generazione del token.

Il JWT contiene:

- username nel campo subject;

- ruoli come claim;
- data di emissione;
- data di scadenza, impostata a dieci ore.

Nelle richieste successive il client invia il token nell'header:

Authorization: Bearer <token>

JwtAuthFilter, implementato come **OncePerRequestFilter**, intercetta ogni richiesta, estrae il token, ricava lo username, ricarica l'utente e verifica firma, identità e scadenza. Se la verifica riesce, inserisce l'autenticazione nel **SecurityContextHolder**.

Il filtro svolge quindi il ruolo di Reference Monitor, perché controlla le richieste prima che raggiungano le operazioni protette.

SecurityConfig stabilisce che:

- il login è pubblico;
- tutte le GET sotto `/api/products/**` sono pubbliche;
- le altre richieste richiedono autenticazione;
- `@PreAuthorize` applica il controllo dei ruoli sui singoli metodi;
- CSRF è disabilitato;
- l'autenticazione di Spring Security è stateless.

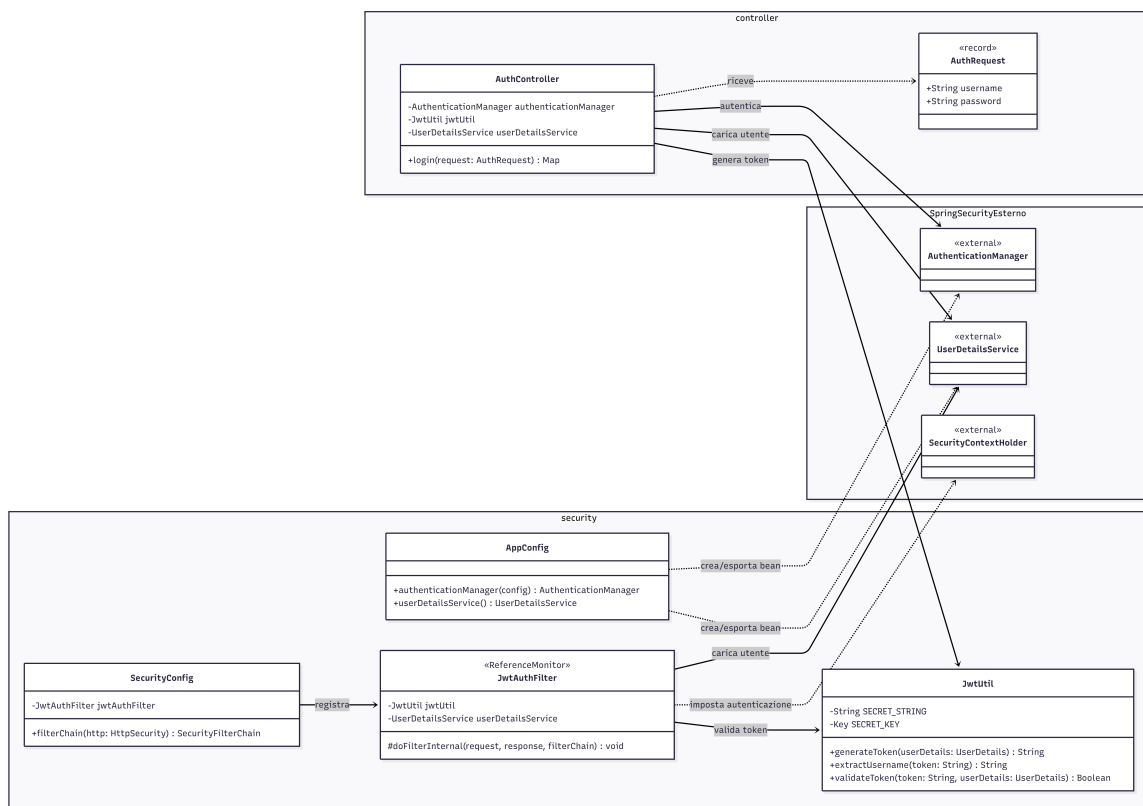


Figura 5: Diagramma UML della sicurezza JWT

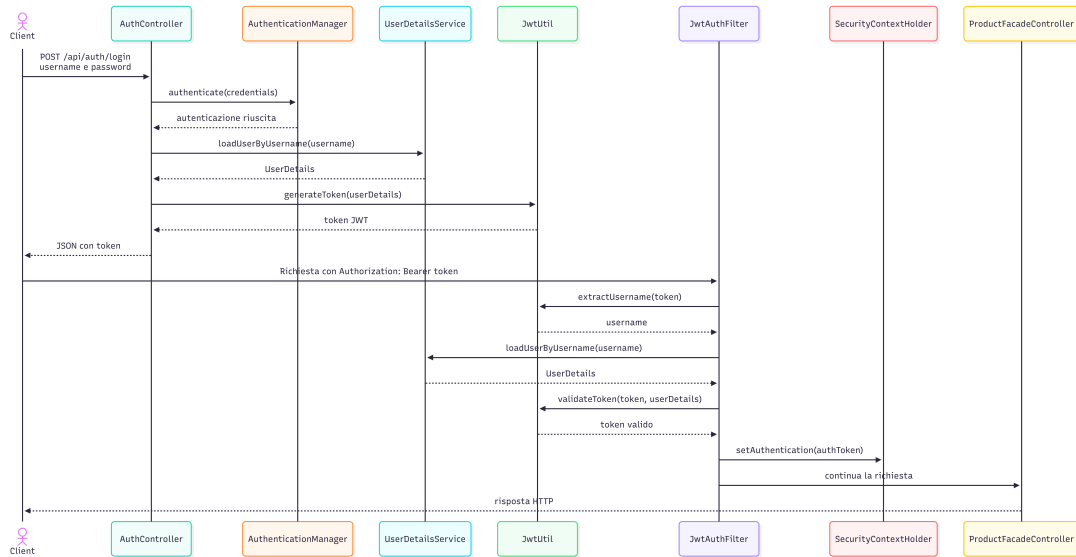


Figura 6: Diagramma di sequenza del login e della validazione JWT

Gli utenti sono configurati in memoria da `AppConfig`:

Username	Password	Ruoli
mario	password	UTENTE
luigi	password	REVISORE
prof	password	AMMINISTRATORE, REVISORE

Tabella 1: Utenti configurati in memoria

Le password in chiaro con `{noop}` sono adeguate soltanto a una dimostrazione didattica. In un sistema reale dovrebbero essere archiviate con hashing, per esempio BCrypt, e la chiave JWT dovrebbe essere esterna al codice.

4 Dettagli di implementazione

4.1 Caricamento dei dati

`ProductRepository` simula un database mediante una `ArrayList<Product>`. Il metodo `init`, annotato con `@PostConstruct`, viene eseguito all'avvio e carica:

- `products.csv`;
- `products2.csv`;
- `fashion.csv`;
- `laptops.csv`.

Apache Commons CSV legge record con strutture differenti. Il repository normalizza nome, prezzo e rating e assegna a ogni prodotto un ID progressivo.

Il parsing del prezzo:

- rimuove simboli di valuta e caratteri non numerici;

- converte la virgola decimale in punto;
- restituisce 0.0 in caso di valore mancante o non valido.

La gestione delle recensioni dipende dal dataset:

- per Fashion vengono estratti fino a tre frammenti testuali;
- per Skincare viene creata una sintesi del numero di recensioni;
- per Laptops viene generata una sintesi tecnica con CPU, RAM e memoria.

Poiché i dati sono conservati solo in memoria, recensioni aggiunte e prezzi modificati vengono persi al riavvio dell'applicazione.

4.2 Prodotti con rating migliore

`ProductService.getTopRatedByCategory`:

1. filtra i prodotti con confronto case-insensitive sulla categoria;
2. ordina per rating decrescente;
3. limita il risultato ai primi cinque;
4. converte ogni `Product` in `ProductDTO`.

Per i laptop il repository assegna attualmente un rating fisso pari a 4.5, perché il relativo dataset non possiede il rating, a differenza del resto dei dataset. Di conseguenza, la classifica dei laptop presenta spesso prodotti con lo stesso rating.

4.3 Prodotto con prezzo migliore

`ProductService.getBestPriceByCategory` filtra i prodotti della categoria e usa `min` con un comparatore sul prezzo. Restituisce un `ProductDTO` oppure `null` se la categoria non contiene prodotti.

Nel batch il singolo prodotto viene inserito in una lista chiamata `bestPrices`, così la struttura del DTO rimane uniforme rispetto alle altre sezioni.

4.4 Prodotti simili

La similarità viene calcolata confrontando i nomi dei prodotti:

1. il nome viene convertito in minuscolo;
2. viene diviso in token alfanumerici;
3. vengono ignorati i token con meno di tre caratteri;
4. si conta il numero di token in comune;
5. vengono mantenuti solo i prodotti con punteggio maggiore di zero;
6. si ordina prima per punteggio di similarità e poi per rating;
7. il risultato viene limitato a dieci prodotti.

La ricerca considera soltanto prodotti appartenenti alla stessa categoria. Quando si parte dall'ID di un prodotto, tale prodotto viene escluso dal risultato. Quando si passa soltanto `productName`, non è possibile identificare univocamente il prodotto di partenza e quindi prodotti con lo stesso nome possono comparire nella lista.

4.5 Recensioni

Le recensioni di un prodotto vengono recuperate tramite:

```
GET /api/products/reviews?id=1
```

Il controller delega a `ProductService.getProductReviews`, che cerca il prodotto per ID e restituisce la lista di recensioni. Se il prodotto non esiste, viene restituita una lista vuota.

L'aggiunta di una recensione avviene tramite:

```
POST /api/products/{id}/reviews
```

Il corpo contiene il testo della recensione. Il metodo è protetto con:

```
@PreAuthorize("hasRole('REVISORE')")
```

4.6 Raggruppamento per fasce di prezzo

`ProductService.getPriceRanges` converte i prodotti in DTO e utilizza `Collectors.groupingBy` per dividerli in tre gruppi:

- prezzo minore di 20: prodotti economici;
- prezzo da 20 incluso a 50 escluso: prodotti medi;
- prezzo maggiore o uguale a 50: prodotti costosi.

Il risultato è una mappa in cui ogni etichetta di fascia è associata alla lista dei relativi prodotti.

4.7 Ricerca e categorie

La ricerca per keyword normalizza la parola in minuscolo e verifica che sia contenuta nel nome del prodotto. La keyword viene inoltre salvata nella `UserSession`.

L'elenco delle categorie viene ottenuto estraendo la categoria da ogni prodotto e applicando `distinct`.

4.8 Modifica del prezzo

La modifica è esposta tramite:

```
PUT /api/products/{id}/price?price=99.99
```

Il metodo modifica direttamente l'oggetto presente nella lista in memoria ed è protetto da:

```
@PreAuthorize("hasRole('AMMINISTRATORE')")
```

L'operazione è idempotente: ripetere la stessa richiesta con lo stesso prezzo produce lo stesso stato finale, poichè la risorsa viene sovrascritta.

5 Elenco delle API

Metodo	Endpoint	Funzione	Accesso
POST	/api/auth/login	Autenticazione e generazione JWT	Pubblico
GET	/api/products/categories	Elenco categorie	Pubblico
GET	/api/products/search?keyword=...	Ricerca per keyword	Pubblico
GET	/api/products/my-history	Cronologia della sessione	Pubblico
GET	/api/products/top-rated?category=...	Primi cinque per rating	Pubblico
GET	/api/products/best-price?category=...	Prezzo migliore	Pubblico
GET	/api/products/similar?category=...	Prodotti simili	Pubblico
GET	/api/products/{id}/similar	Prodotti simili da ID	Pubblico
GET	/api/products/suggestions?...	Request Batch	Pubblico
GET	/api/products/reviews?id=...	Recensioni del prodotto	Pubblico
GET	/api/products/price-ranges	Raggruppamento per prezzo	Pubblico
GET	/api/products/{id}	Dettaglio interno per debug	AMMINISTRATORE
POST	/api/products/{id}/reviews	Aggiunta recensione	REVISORE
PUT	/api/products/{id}/price?...	Modifica prezzo	AMMINISTRATORE

6 Esempio di utilizzo

6.1 Login

```
POST http://localhost:8080/api/auth/login
Content-Type: application/json

{
  "username": "luigi",
  "password": "password"
}
```

Risposta:

```
{
  "token": "<token JWT>"
}
```

6.2 Richiesta protetta

```
POST http://localhost:8080/api/products/1/reviews
Authorization: Bearer <token JWT>
Content-Type: text/plain

Ottimo prodotto
```

6.3 Request Batch

```
GET http://localhost:8080/api/products/suggestions?category=Laptops&productName=Apple%20MacBook%20Pro
```

La risposta contiene contemporaneamente i cinque laptop con rating migliore, il laptop con prezzo minore e fino a dieci prodotti simili al nome indicato.

7 Test e avvio

Il progetto usa Maven e può essere verificato con:

```
.\mvnw.cmd test
```

Oppure, se si vuole lanciare una specifica classe di test:

```
.\mvnw.cmd -Dtest=SecurityTests test
.\mvnw.cmd -Dtest=ProductFacadeControllerTests test
.\mvnw.cmd -Dtest=ProductServiceTests test
```

Sono presenti test automatici basati su JUnit, Spring Boot Test e MockMvc. MockMvc permette di simulare richieste HTTP ai controller senza avviare manualmente il server sulla porta 8080.

Classe	Scopo
ApplicationTests	verifica il corretto caricamento del contesto Spring
ProductServiceTests	verifica logica applicativa, categorie, rating, prezzo migliore, prodotti simili e prodotto inesistente
ProductFacadeControllerTests	verifica alcuni endpoint pubblici e la Request Batch con due parametri
SecurityTests	verifica login JWT, ruoli, endpoint protetti, aggiunta recensioni, modifica prezzo e dettaglio admin

Tabella 3: Classi di test automatico

L'esecuzione attuale produce 15 su 15 test superati, senza fallimenti o errori.

In `application.properties` è configurato lo shutdown graduale. Alla chiusura Spring Boot completa le richieste in corso entro il timeout configurato e rilascia la porta HTTP.

8 Limiti e possibili miglioramenti

L'implementazione soddisfa i requisiti didattici principali. Di seguito ogni limite è associato alla possibile evoluzione che permetterebbe di superarlo:

1. **Persistenza e concorrenza.** I dati sono conservati in una lista in memoria: le modifiche vengono perse al riavvio e la gestione di accessi concorrenti è limitata. Come evoluzione si potrebbe introdurre un database reale.
2. **Rappresentazione dei dati.** Lo stesso `ProductDTO` viene utilizzato in diversi contesti, mentre l'endpoint amministrativo di debug restituisce direttamente `Product`. Si potrebbero adottare DTO distinti per liste, dettagli e richieste di modifica.

3. **Gestione degli errori HTTP.** Alcuni metodi restituiscono `null`, valori predefiniti o liste vuote quando una risorsa non esiste. Si potrebbero usare `ResponseEntity`, eccezioni personalizzate e `@ControllerAdvice` per restituire risposte standard, per esempio 404 Not Found.
4. **Segreto JWT nel codice.** La chiave usata per firmare i token è definita direttamente nel sorgente e può essere letta da chi accede al progetto. Dovrebbe essere salvata in una variabile d'ambiente o in una configurazione esterna.
5. **Password in chiaro.** Gli utenti di esempio usano password con `{noop}`, quindi prive di cifratura o hashing. In una versione reale si dovrebbe usare BCrypt.
6. **Assegnazione dei ruoli.** `prof` riceve direttamente sia `AMMINISTRATORE` sia `REVISORE`. Questa scelta è stata la scelta più rapida, però una gerarchia dei ruoli permetterebbe di stabilire una regola generale, per esempio: `[AMMINISTRATORE > REVISORE > UTENTE]`.
7. **Copertura dei test.** Sono stati aggiunti test automatici per service, controller e sicurezza. Una possibile evoluzione ulteriore sarebbe aumentare la copertura sui casi limite, per esempio token scaduto, prodotto inesistente con risposta 404, concorrenza sulle modifiche e validazione dei dati in input.
8. **Rating dei laptop.** Tutti i laptop ricevono un rating fisso pari a 4.5, rendendo poco significativa la classifica dei prodotti migliori. Come evoluzione si potrebbe usare un dataset contenente rating reali oppure calcolare un indicatore a partire da altre informazioni.

9 Conclusioni

Il progetto realizza un backend REST organizzato secondo una struttura a livelli e applica diversi pattern tipici dei sistemi distribuiti. La Remote Facade concentra l'accesso alle funzionalità, i DTO controllano il trasferimento dei dati, il Session State conserva la cronologia delle ricerche, la Request Batch aggrega più risultati e il sistema JWT con controllo RBAC protegge le operazioni di scrittura.

Il progetto dimostra quindi l'integrazione tra progettazione delle API, gestione dei dati, sicurezza e applicazione di design pattern. La natura in-memory e alcune semplificazioni sono adatte a un contesto didattico, mentre gli interventi indicati permetterebbero di evolverlo verso un sistema più robusto e vicino a un ambiente di produzione.